

A Deep Dive into Rainbow DQN

Chapter 3: Dueling Network Architectures

Harsh Verma

June 26, 2025

Abstract

Standard Deep Q-Network architectures utilize a single stream of fully-connected layers to output Q-values for all actions. This design conflates the value of being in a particular state with the value of taking a specific action in that state. The Dueling Network Architecture, proposed by Wang et al. (2016), provides a more effective representation by explicitly disentangling these two quantities. It features two separate streams for estimating the state-value function and the action-advantage function, which are then combined to produce the final Q-values. This chapter explores the theory, mathematical formulation, and implementation of this architecture, highlighting its role in improving learning efficiency and generalization.

1 Motivation: A More Efficient Value Representation

1.1 The Inefficiency of a Single-Stream Architecture

A conventional DQN passes its learned features into a single sequence of layers that must produce a specific Q-value for every action. This structure has an inherent representational bottleneck.

Consider an agent learning to drive. The value of any state where a collision is imminent is extremely low. A standard DQN must learn this fact separately for every possible action it could take in that state (e.g., the Q-value for "turning left into a wall" is low, and the Q-value for "accelerating into a wall" is also low). This is redundant. The state itself is bad, independent of the action.

The Dueling Network architecture is designed to solve this by creating separate pathways for the network to reason about state-dependent values and action-dependent values.

2 The Dueling Architecture: Theory and Formulation

2.1 Decomposing the Q-Function

The core theoretical idea is to decompose the Q-value into two distinct components: a **State-Value** and an **Action-Advantage**.

Definition 2.1 (State-Value Function, $V(s)$). *The state-value function represents the intrinsic value of being in state s . It answers the question, "How good is it, on average, to be in this situation?" This value is a property of the state alone. For the driving agent, $V(s)$ would be very low for any state immediately preceding a crash.*

Definition 2.2 (Action-Advantage Function, $A(s, a)$). *The action-advantage function represents the relative importance of taking action a compared to all other actions in state s . It*

answers the question, "Given the situation, how much better is this specific action than the other available options?" By definition, for the optimal action a^* , the advantage $A(s, a^*)$ is positive, while for suboptimal actions, the advantage is negative.

2.2 The "How" and "Why" of the Two Streams

The network architecture mirrors this decomposition. After an initial set of shared feature-learning layers (e.g., convolutional layers), the network splits into two streams:

- **The Value Stream** learns to produce a single scalar output, $V(s)$. Because its output is independent of the action, it can be updated from the TD-error of *any* transition starting from state s . This makes the learning of state values extremely data-efficient.
- **The Advantage Stream** learns to produce a vector of outputs, one for each action, representing $A(s, a)$. This stream learns to focus only on the aspects of the state that make one action preferable to another. In many states where the choice of action is irrelevant, this stream can learn to output values close to zero for all actions.

2.3 The Identifiability Problem and Its Solution

A naive combination $Q = V + A$ is mathematically ambiguous (unidentifiable), as discussed previously. To solve this, we must constrain the advantage stream to anchor the decomposition. The most stable and widely-used method is to force the advantages for a state to sum to zero by subtracting their mean:

$$Q(s, a; \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a'; \theta, \alpha) \right) \quad (1)$$

This formulation has a powerful side effect: it gives a clear identity to each component. $V(s)$ now directly learns the mean value of the state, and $A(s, a)$ learns the relative deviations from that mean for each action. This provides stable learning targets for both streams.

3 Implementation and Visualization

3.1 Architectural Diagram

Figure 1 illustrates the information flow in a Dueling Network compared to a standard DQN.

3.2 PyTorch Code Implementation

The following demonstrates how to implement this architecture in PyTorch.

```

1 import torch
2 import torch.nn as nn
3
4 class DuelingDQN(nn.Module):
5     def __init__(self, input_shape, n_actions):
6         super(DuelingDQN, self).__init__()
7         # ... (convolutional layers and helper functions as before) ...
8
9         # State-Value Stream
10        self.value_stream = nn.Sequential(
11            nn.Linear(conv_out_size, 512),
12            nn.ReLU(),
13            nn.Linear(512, 1) # Outputs a single scalar: V(s)
14        )
15
```

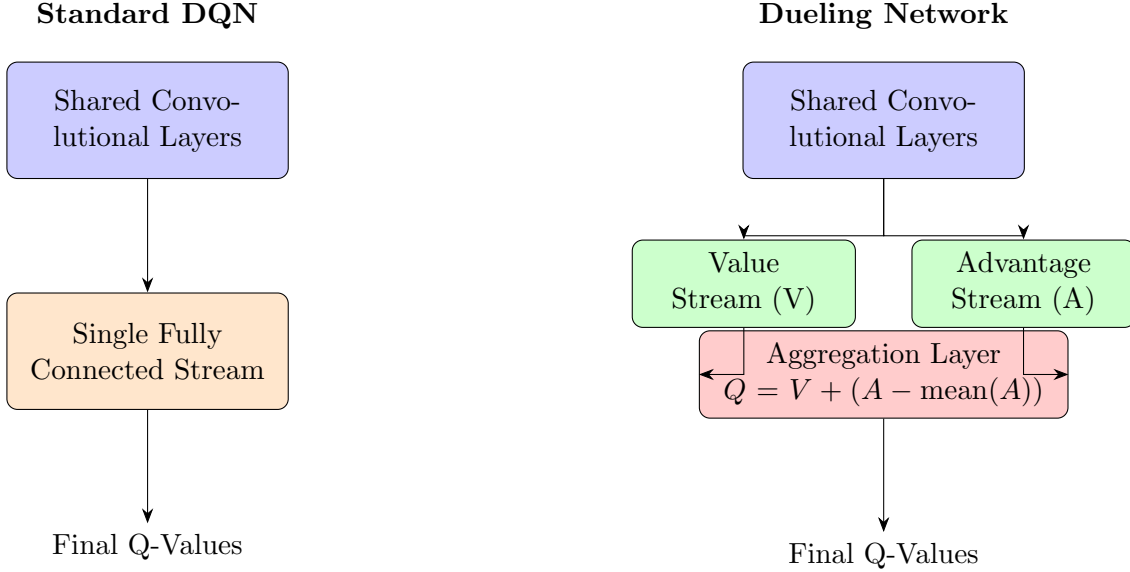


Figure 1: A comparison of a Standard DQN architecture (left) versus the Dueling Network architecture (right). The dueling network splits its computation into two streams to estimate state-values and action-advantages separately before combining them.

```

16     # Action-Advantage Stream
17     self.advantage_stream = nn.Sequential(
18         nn.Linear(conv_out_size, 512),
19         nn.ReLU(),
20         nn.Linear(512, n_actions) # Outputs a value for each action: A(s,a)
21     )
22
23     def forward(self, state):
24         features = self.conv_layers(state).view(state.size(0), -1)
25
26         # Get state-value and advantages from their respective streams
27         state_value = self.value_stream(features)
28         advantages = self.advantage_stream(features)
29
30         # Combine them using the mean-aggregation formula from Eq. 2
31         q_values = state_value + (advantages - advantages.mean(dim=1, keepdim=
32                                True))
33         return q_values

```

Listing 1: PyTorch implementation of a Dueling Network.

4 Discussion and Deeper Insights

4.1 Intuitive Evidence: Saliency Maps

The original paper provided compelling visual evidence using saliency maps, which highlight the pixels the network “pays attention to.” They found that:

- The **Value Stream** learned to focus on long-term indicators of value, like the score on the screen or the horizon line in a driving game.
- The **Advantage Stream** learned to ignore these and focus only on what was immediately relevant for action—the ball and paddle in Pong, or nearby enemy ships.

This provides strong intuitive proof that the two streams successfully learn to specialize, validating the entire architectural motivation.

4.2 Synergy with Other Components

The Dueling architecture is highly synergistic. With Prioritized Replay, the more robust value estimates can lead to more meaningful TD-errors and better prioritization. It is also directly compatible with a Distributional approach, where the value and advantage streams are modified to output full distributions instead of scalar values, as is done in the full Rainbow agent.